Lab 6 — Code and Branch Coverage Report

Module under test: `ski_resort_pass.py` — `process_resort_bookings()` and `check_chairlift_anemometer()` (59 statements, 42 branch arcs)

How to reproduce

bash

```
pip install coverage pytest
cd "Lab 6"

# Code (line) coverage
coverage run -m pytest CodeCoverageTest_ski_resort_pass_application.py
coverage report -m

# Branch coverage
coverage run --branch -m pytest BranchCoverageTest_ski_resort_pass_application.
coverage report -m
```

1. Starting point (starter `test_ski_resort.py`, 1 test)

Metric	Result
Line coverage of `ski_resort_pass.py`	**47 %** (31 of 59 statements never run)
Branch coverage of `ski_resort_pass.py`	**44 %** (42 branch arcs, 12 only partly taken)
Never executed	guard clause, inactive groups, season / half-day / unknown durations, child / senior pricing, rejected ages, blackout handling, gear rental, locker fee, `AUDIT_REVIEW`, `ZERO_SALES`, and the whole of `check_chairlift_anemometer`

2. Code coverage suite — `CodeCoverageTest_ski_resort_pass_application.py`

Terminal output (`python -m coverage run -m pytest …` then `python -m coverage report -m`):

Code coverage: 17 passed, 100% line coverage

Metric	Result
Tests	17, all passing
Line coverage of `ski_resort_pass.py`	**100** % (59 / 59 statements, 0 missed)

Analysis. Line coverage only asks whether a statement ran, so the suite has one focused test per outcome: the `ValueError` guard, an empty and an inactive booking list, each of the three pass durations plus an unknown one, child / senior / adult pricing, a rejected age, blackout override on a full-day and a half-day pass, gear rental, the lodge locker, all three status codes, and the three anemometer results (`NO_DATA` , `SUSPEND_LIFT` , `SAFE_OPERATION`). The expected values were worked out by hand from the pricing rules (for example child full-day = 80 × 0.60 = 48), not copied from the program's output.

3. Branch coverage suite — `BranchCoverageTest_ski_resort_pass_application.py`

Terminal output (`python -m coverage run --branch -m pytest …` then `python -m coverage report -m`):

Branch coverage: 89 passed, 100% branch coverage

Metric	Result
Tests	89 (17 carried over + 72 new, parametrised), all passing
Line coverage of `ski_resort_pass.py`	**100** % (59 / 59)
Branch coverage of `ski_resort_pass.py`	**100** % (42 / 42 branch arcs, 0 partial)

What was added. The file is self-contained (it keeps the code-coverage tests) and adds tests so that every decision is taken on *both* its True and False side:

- **Guard clause** `not isinstance(groups, list) or facility_fee < 0` : each operand tested on its own (`None` , `str` , `tuple` , `dict` , `int` as groups; `-0.01` , `-1` , `-100` as fee) and the `fee == 0` boundary.
- **`.get()` defaults:** missing `is_active` , `duration` , `skiers` , `age` , `gear_rental` , `blackout_override` .
- **Duration** `if / elif / else` **:** all arms, plus near-misses such as `"FULL_DAY"` , `"half-day"` , `""` , `None` .
- **Age boundaries:** `-1 / 0 / 1` , `11 / 12` , `64 / 65` , `100` .
- **Blackout override:** half-day `break` vs other durations, priority over the child / senior discount, interaction with gear rental and the locker fee, and `break` voiding only its own group.
- **Status decisions:** exactly 4 vs 5 rejections (`> 4`), and `AUDIT_REVIEW` taking priority over `ZERO_SALES` .
- **Rounding:** `10.126 → 90.13` and `10.124 → 90.12` .

- **Anemometer:** `45` vs `46` mph, `>= 2` streak, streak reset by a calm reading, streak *not* carried from one lift to the next, empty lifts, `None` input.

Analysis — why a second suite was worth writing. coverage.py measures branches as line-to-line jumps, and the line-oriented suite already took every jump, so it also scores 100 % branch coverage. A percentage alone therefore cannot tell the two suites apart. To check whether the assertions are actually strong, 27 single-edit bugs ("mutants") were applied one at a time to a scratch copy of the module:

Suite	Mutants caught	Tests
`CodeCoverageTest_…`	14 / 27	17
`BranchCoverageTest_…`	**27 / 27**	89

The 13 mutants the line-oriented suite missed were all boundary, default-value or priority bugs: `facility_fee < 0` → `<= 0`; `age < 12` → `<= 12`; `age >= 65` → `> 65`; `rejected_passes > 4` → `>= 4`; the audit check ignoring zero-revenue runs; `speed > 45` → `>= 45`; the gust streak carrying over between lifts; changed defaults for `duration`, `is_active`, `includes_lodge_locker` and `age`; rounding to 1 dp instead of 2; and `if wind_streams == []` instead of `if not wind_streams`. These decisions are executed in both directions by the line suite but never at the value where they flip, which is exactly what the branch suite targets.

4. Module behaviour worth confirming against the spec

These are pinned by tests as the *current* behaviour; they may or may not be intended:

1. A blackout-override skier is counted as a **rejected pass** yet is **still billed 50 %** of the base rate (on non-half-day passes).
2. On a half-day pass a blackout skier voids the group total, but the **locker fee is still added afterwards**, so a "voided" group can still produce revenue equal to `facility_fee`.
3. An unknown-duration group is skipped before the locker decision, so it never pays the locker fee.